

Week-5

Q1: What are the key limitations of traditional MapReduce that make Spark a preferred choice for modern big data processing?

MapReduce has a few limitations that Spark was built to fix:

- Stores intermediate results on disk after every step, so it involves a lot of disk I/O.
- Slow for iterative algorithms (like ML), since data has to be re-read from disk each time.
- Only supports batch processing, no streaming.
- Needs a lot of boilerplate code for even simple logic.
- No built-in SQL, streaming, or ML support.

Spark fixes this mainly through in-memory computing (10-100x faster), simpler APIs in Python/Scala/Java/R, built-in SQL/Streaming/MLlib/GraphX, and fault tolerance through RDD lineage instead of relying only on replication.

Q2: Explain how Spark uses In-Memory Computing to speed up iterative machine learning algorithms compared to disk-based systems.

In disk-based systems like MapReduce, every stage reads from disk and writes back to disk. For iterative algorithms (gradient descent, PageRank, etc.) the same data gets scanned again and again, so this disk cost keeps repeating.

- Spark loads the dataset into RAM once (using `.cache()` or `.persist()`) and reuses it across iterations.
- RAM access is much faster than disk, so repeated computations skip the disk bottleneck.
- Spark's DAG engine also plans the transformations before running them, avoiding unnecessary recomputation.
- If memory runs out, only the extra data spills to disk instead of the whole job failing.

So an iterative ML loop that would hit disk every round in MapReduce just reuses one in-memory copy in Spark, which is why it's reported as 10-100x faster.

Q3: Write a code snippet to remove all duplicate rows from a DataFrame based on a specific set of columns: `user_id` and `transaction_date`.

Code:

```
week5_assignment - Visual Studio Code
week5_q3.py x
1  from pyspark.sql import SparkSession
2  from pyspark.sql.types import StructType, StructField, StringType, IntegerType, DoubleType
3
4  spark = SparkSession.builder.appName("week5").getOrCreate()
5
6  schema = StructType([
7      StructField("user_id", IntegerType()),
8      StructField("transaction_date", StringType()),
9      StructField("amount", DoubleType()),
10 ])
11 data = [
12     (101, "2026-01-05", 250.0),
13     (101, "2026-01-05", 250.0),
14     (102, "2026-01-06", 90.0),
15     (103, "2026-01-06", 300.0),
16     (103, "2026-01-07", 300.0),
17     (104, "2026-01-08", 150.0),
18     (104, "2026-01-08", 175.0),
19 ]
20 df = spark.createDataFrame(data, schema)
21
22 print("Original data:")
23 df.show(truncate=False)
24
25 df_clean = df.dropDuplicates(["user_id", "transaction_date"])
26
27 print("After removing duplicates:")
28 df_clean.show(truncate=False)
29
30 spark.stop()
```

Output:

```
TERMINAL bash
student@LAPTOP-7QK2 MINGW64 ~/week5_assignment (main)
$ python week5_q3.py
Original data:

+-----+-----+-----+
|user_id|transaction_date|amount|
+-----+-----+-----+
|101    |2026-01-05      |250.0 |
|101    |2026-01-05      |250.0 |
|102    |2026-01-06      |90.0  |
|103    |2026-01-06      |300.0 |
|103    |2026-01-07      |300.0 |
|104    |2026-01-08      |150.0 |
|104    |2026-01-08      |175.0 |
+-----+-----+-----+

After removing duplicates:

+-----+-----+-----+
|user_id|transaction_date|amount|
+-----+-----+-----+
|103    |2026-01-06      |300.0 |
|101    |2026-01-05      |250.0 |
|104    |2026-01-08      |150.0 |
|102    |2026-01-06      |90.0  |
|103    |2026-01-07      |300.0 |
+-----+-----+-----+
```

Q4: Given a DataFrame df_sales, write a query to filter for rows where the region is 'West' and then group by product_category to find the average sale_amount.

Code:

```
week5_assignment - Visual Studio Code
week5_q4.py x
1 from pyspark.sql import SparkSession
2 from pyspark.sql.functions import avg
3
4 spark = SparkSession.builder.appName("week5").getOrCreate()
5
6 data = [
7     ("West", "Electronics", 200.0),
8     ("West", "Electronics", 400.0),
9     ("West", "Grocery", 50.0),
10    ("East", "Electronics", 999.0),
11    ("West", "Grocery", 70.0),
12    ("West", "Clothing", 120.0),
13    ("South", "Grocery", 30.0),
14    ("West", "Clothing", 160.0),
15 ]
16 df_sales = spark.createDataFrame(data, ["region", "product_category", "sale_amount"])
17
18 result = (df_sales.filter(df_sales.region == "West")
19           .groupBy("product_category")
20           .agg(avg("sale_amount").alias("avg_sale_amount")))
21
22 print("Average sale_amount by category (region = West):")
23 result.show(truncate=False)
24
25 spark.stop()
```

output:

```
TERMINAL bash
student@LAPTOP-7QK2 MINGW64 ~/week5_assignment (main)
$ python week5_q4.py
Average sale_amount by category (region = West):

+-----+-----+
|product_category|avg_sale_amount|
+-----+-----+
|Grocery         |60.0           |
|Electronics     |300.0          |
|Clothing        |140.0          |
+-----+-----+
```

Q5: What is the difference between `.na.drop()` and `.na.fill()`? Provide a code example of filling null values in a status column with the string 'Unknown'.

- `.na.drop()` removes rows that have nulls. By default it drops a row if any column is null (can be changed with `how="all"` or `subset=[...]`).
- `.na.fill()` replaces the null with a value instead of deleting the row, so you don't lose data — only the missing value gets filled in.

Code:

```
week5_assignment - Visual Studio Code
week5_q5.py x
1  from pyspark.sql import SparkSession
2
3  spark = SparkSession.builder.appName("week5").getOrCreate()
4
5  data = [(1, "Shipped"), (2, None), (3, "Delivered"), (4, None), (5, "Pending")]
6  df = spark.createDataFrame(data, ["order_id", "status"])
7
8  print("Before filling nulls:")
9  df.show(truncate=False)
10
11 df_filled = df.na.fill({"status": "Unknown"})
12
13 print("After filling nulls:")
14 df_filled.show(truncate=False)
15
16 spark.stop()
```

Output:

```
TERMINAL bash
student@LAPTOP-7QK2 MINGW64 ~/week5_assignment (main)
$ python week5_q5.py
Before filling nulls:

+-----+-----+
|order_id|status  |
+-----+-----+
|1       |Shipped |
|2       |NULL    |
|3       |Delivered|
|4       |NULL    |
|5       |Pending |
+-----+-----+

After filling nulls:
+-----+-----+
|order_id|status  |
+-----+-----+
|1       |Shipped |
|2       |Unknown |
|3       |Delivered|
|4       |Unknown |
|5       |Pending |
+-----+-----+
```

Q6: Write a query to find the total count of records for each city in a DataFrame, but only for cities where the count is greater than 100.

Code:

```
week5_assignment - Visual Studio Code
week5_q6.py x
1  from pyspark.sql import SparkSession
2  from pyspark.sql.functions import count
3
4  spark = SparkSession.builder.appName("week5").getOrCreate()
5
6  data = [("Delhi",)]*150 + [("Mumbai",)]*80 + [("Dehradun",)]*120 + [("Pune",)]*40
7  df = spark.createDataFrame(data, ["city"])
8
9  result = (df.groupBy("city")
10             .agg(count("*").alias("total_count"))
11             .filter("total_count > 100"))
12
13  print("Cities with more than 100 records:")
14  result.show(truncate=False)
15
16  spark.stop()
```

Output:

```
TERMINAL bash
student@LAPTOP-7QK2 MINGW64 ~/week5_assignment (main)
$ python week5_q6.py
Cities with more than 100 records:
+-----+-----+
|city    |total_count|
+-----+-----+
|Dehradun|120        |
|Delhi   |150        |
+-----+-----+
```

Q7: How does the immutability of Spark DataFrames affect how you perform "data cleaning" steps like dropping columns or renaming them?

Spark DataFrames can't be changed in place. Any cleaning step (dropping a column, renaming, filtering, casting) returns a new DataFrame instead of modifying the original.

- So things like `df.drop("col")` or `df.withColumnRenamed(...)` have to be reassigned — e.g. `df = df.drop("col")` — otherwise the change doesn't stick.
- This makes the pipeline safer since earlier stages never get silently overwritten by later ones.
- It also ties into lazy evaluation — each transformation is just a step in a DAG until an action triggers it, which is why chaining many cleaning steps isn't as wasteful as it sounds.

Q8: Write a Spark command to filter a dataset for rows where the age is between 18 and 30 (inclusive) and the subscription is 'Premium'.

Code:

```
week5_assignment - Visual Studio Code
week5_q8.py x
1  from pyspark.sql import SparkSession
2
3  spark = SparkSession.builder.appName("week5").getOrCreate()
4
5  data = [
6      ("Aarav", 22, "Premium"),
7      ("Diya", 17, "Premium"),
8      ("Kabir", 29, "Free"),
9      ("Meera", 31, "Premium"),
10     ("Ishaan", 18, "Premium"),
11     ("Zara", 30, "Premium"),
12 ]
13 df = spark.createDataFrame(data, ["name", "age", "subscription"])
14
15 print("Original users:")
16 df.show(truncate=False)
17
18 result = df.filter((df.age >= 18) & (df.age <= 30) & (df.subscription == "Premium"))
19
20 print("Age 18-30 and Premium subscribers:")
21 result.show(truncate=False)
22
23 spark.stop()
```

Output:

```
TERMINAL
$ python week5_q8.py
Original users:
+-----+
|name|age|subscription|
+-----+
|Aarav|22|Premium|
|Diya|17|Premium|
|Kabir|29|Free|
|Meera|31|Premium|
|Ishaan|18|Premium|
|Zara|30|Premium|
+-----+

Age 18-30 and Premium subscribers:
+-----+
|name|age|subscription|
+-----+
|Aarav|22|Premium|
|Ishaan|18|Premium|
|Zara|30|Premium|
+-----+
```

Q9: When cleaning a dataset, why is it often better to handle null values before performing mathematical aggregations like sum() or avg()?

- Spark's aggregations ignore nulls by default, which can quietly skew an avg() if nulls were meant to represent zero.
- count(column) skips nulls but count("*") doesn't, so mixing these without deciding how to treat nulls gives inconsistent numbers.
- Nulls can also break later joins or casts if they're not cleaned up early.
- Deciding upfront whether a null means zero, unknown, or should be dropped makes the aggregation meaning clear and reproducible instead of relying on Spark's default behavior.

Q10: Write the code to revise a column named raw_timestamp by casting it to a TimestampType and renaming it to event_time.

Code:

```
week5_assignment - Visual Studio Code
week5_q10.py x
1  from pyspark.sql import SparkSession
2  from pyspark.sql.functions import col
3  from pyspark.sql.types import TimestampType
4
5  spark = SparkSession.builder.appName("week5").getOrCreate()
6
7  data = [("2026-01-05 10:15:00",), ("2026-01-06 08:30:00",), ("2026-01-07 21:45:10",)]
8  df = spark.createDataFrame(data, ["raw_timestamp"])
9
10 print("Original schema:")
11 df.printSchema()
12
13 df = (df.withColumn("event_time", col("raw_timestamp").cast(TimestampType()))
14       .drop("raw_timestamp"))
15
16 print("After casting and renaming:")
17 df.show(truncate=False)
18 df.printSchema()
19
20 spark.stop()
```

Output:

```
TERMINAL bash

student@LAPTOP-7QK2 MINGW64 ~/week5_assignment (main)
$ python week5_q10.py
Original schema:
root
|-- raw_timestamp: string (nullable = true)

After casting and renaming:

+-----+
|event_time      |
+-----+
|2026-01-05 10:15:00|
|2026-01-06 08:30:00|
|2026-01-07 21:45:10|
+-----+

root
|-- event_time: timestamp (nullable = true)
```


Q11: Explain the "Shuffle" process that occurs during a grouping operation. Why is it considered a wide transformation?

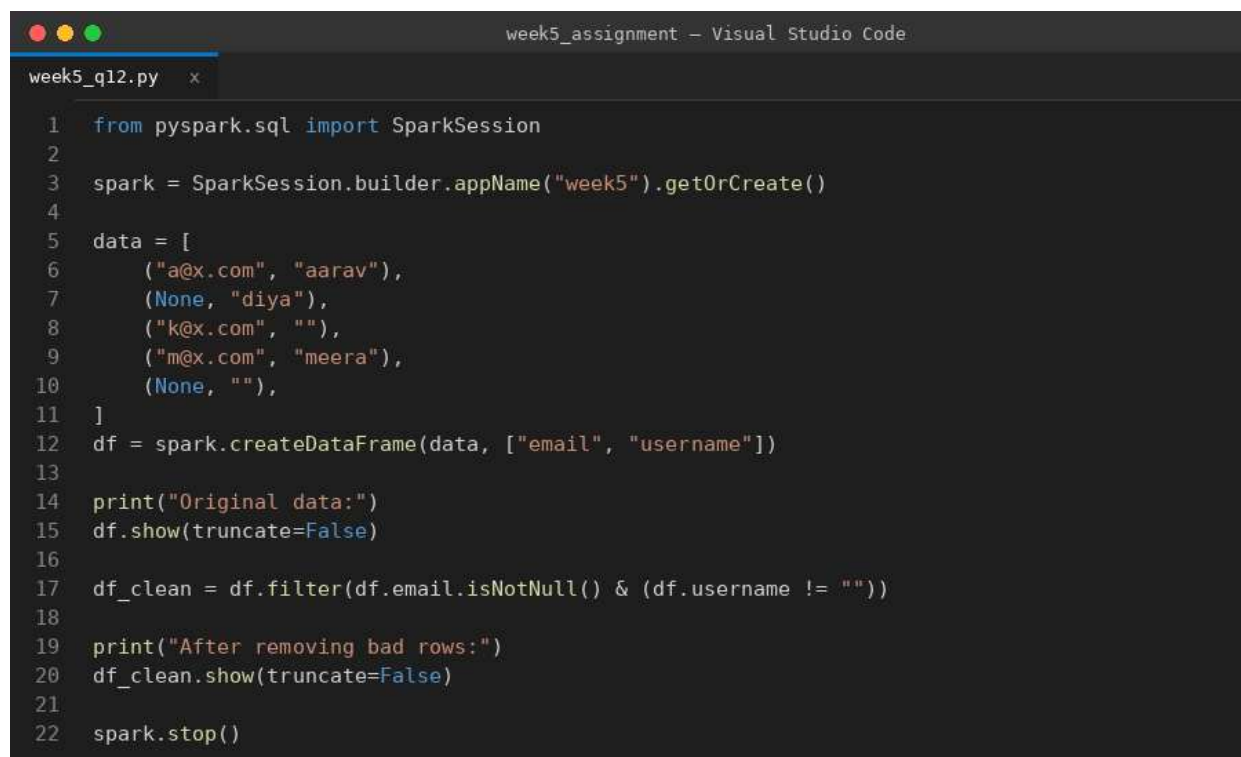
A shuffle is when Spark redistributes data across partitions/nodes so that rows with the same key end up together.

- During groupBy() (or join, distinct, repartition), Spark needs all rows with the same key in one place before it can aggregate them.
- Since matching rows can start out on different partitions or machines, Spark writes data to disk and moves it over the network to regroup it by key — that's the shuffle.
- This is expensive because of the disk I/O, serialization, and network transfer involved.

It's called a wide transformation because one output partition can depend on data from many input partitions — unlike a narrow transformation (map, filter) where each output partition only needs one input partition and no data movement.

Q12: Write a code snippet that identifies and removes rows where the email column contains null values OR the username is an empty string.

Code:



```
week5_assignment - Visual Studio Code
week5_q12.py x
1  from pyspark.sql import SparkSession
2
3  spark = SparkSession.builder.appName("week5").getOrCreate()
4
5  data = [
6      ("a@x.com", "aarav"),
7      (None, "diya"),
8      ("k@x.com", ""),
9      ("m@x.com", "meera"),
10     (None, ""),
11 ]
12 df = spark.createDataFrame(data, ["email", "username"])
13
14 print("Original data:")
15 df.show(truncate=False)
16
17 df_clean = df.filter(df.email.isNotNull() & (df.username != ""))
18
19 print("After removing bad rows:")
20 df_clean.show(truncate=False)
21
22 spark.stop()
```

Output:

```
TERMINAL bash

student@LAPTOP-7QK2 MINGW64 ~/week5_assignment (main)
$ python week5_q12.py
Original data:

+-----+-----+
|email  |username|
+-----+-----+
|a@x.com|aarav   |
|NULL   |diya    |
|k@x.com|        |
|m@x.com|meera   |
|NULL   |        |
+-----+-----+

After removing bad rows:

+-----+-----+
|email  |username|
+-----+-----+
|a@x.com|aarav   |
|m@x.com|meera   |
+-----+-----+
```

Q13: How do you use the .agg() function to calculate multiple statistics at once, such as the min, max, and mean of the price column?

Code:

```
week5_assignment - Visual Studio Code

week5_q13.py x
1 from pyspark.sql import SparkSession
2 from pyspark.sql.functions import min as spark_min, max as spark_max, mean
3
4 spark = SparkSession.builder.appName("week5").getOrCreate()
5
6 data = [(120.0,), (450.0,), (80.0,), (999.5,), (300.0,)]
7 df = spark.createDataFrame(data, ["price"])
8
9 result = df.agg(
10     spark_min("price").alias("min_price"),
11     spark_max("price").alias("max_price"),
12     mean("price").alias("mean_price"),
13 )
14
15 print("Price statistics:")
16 result.show(truncate=False)
17
18 spark.stop()
```

Output:

```
TERMINAL bash
student@LAPTOP-7QK2 MINGW64 ~/week5_assignment (main)
$ python week5_q13.py
Price statistics:

+-----+-----+-----+
|min_price|max_price|mean_price|
+-----+-----+-----+
|80.0      |999.5      |389.9      |
+-----+-----+-----+
```

Q14: In the context of cleaning a dataset, what is the risk of using inferSchema=true when your source data contains messy or inconsistent date formats?

- Spark only samples the data to guess types, so mixed date formats (e.g. "2024-01-05" vs "05/01/2024" vs "N/A") can make it fall back to StringType instead of an actual date type.
- Rows that don't match the guessed format can silently turn into null later, which is easy to miss.
- It also adds an extra pass over the data before loading, which costs time on large files.
- If the sample doesn't cover all the formats in the data, the inferred schema can even differ between runs.

It's usually safer to define the schema explicitly, or load dates as strings first and parse them with to_date()/to_timestamp() using a known format.

Q15: Write a final processing pipeline that: filters out duplicates, fills null prices with 0, groups by store_id to calculate total revenue.

Code:

```
week5_assignment - Visual Studio Code
week5_q15.py x
1 from pyspark.sql import SparkSession
2 from pyspark.sql.functions import sum as spark_sum
3
4 spark = SparkSession.builder.appName("week5").getOrCreate()
5
6 data = [
7     ("S1", 100.0),
8     ("S1", 100.0),
9     ("S1", 50.0),
10    ("S2", None),
11    ("S2", 200.0),
12    ("S3", 75.0),
13    ("S3", 25.0),
14 ]
15 df = spark.createDataFrame(data, ["store_id", "price"])
16
17 print("Raw data:")
18 df.show(truncate=False)
19
20 result = (df.dropDuplicates()
21           .na.fill({"price": 0})
22           .groupBy("store_id")
23           .agg(spark_sum("price").alias("total_revenue"))
24           .orderBy("store_id"))
25
26 print("Total revenue by store:")
27 result.show(truncate=False)
28
29 spark.stop()
```

Output:

```
TERMINAL bash
student@LAPTOP-7QK2 MINGW64 ~/week5_assignment (main)
$ python week5_q15.py
Raw data:

+-----+-----+
|store_id|price|
+-----+-----+
|S1      |100.0|
|S1      |100.0|
|S1      |50.0 |
|S2      |NULL |
|S2      |200.0|
|S3      |75.0 |
|S3      |25.0 |
+-----+-----+

Total revenue by store:

+-----+-----+
|store_id|total_revenue|
+-----+-----+
|S1      |150.0        |
|S2      |200.0        |
|S3      |100.0        |
+-----+-----+
```